SOFTWARE REQUIREMENT SPECIFICATION

FOR

AUCTION MANAGEMENT SYSTEM

Under the Subject of

Software Engineering

(Semester – I)

Submitted by

NAGORWALA MOHAMMED ATIF ASIF(25002170210058)
PATEL KRISH RAJESHBHAI(25002170210075)
DOSHI RUTVI PINTALKUMAR(25002170210026)

Ms.Nidhi Mehta
(Faculty Guide)

Dr. Meghna Parikh
(FY2-Head of Department)

Academic Year
2025 – 26

L. J. Institute of Engineering & Technology, Ahmedabad (LJU)

L. J. INSTITUTE OF ENGINEERING AND TECHNOLOGY

FY2-Department 2025 – 26

CERTIFICATE

Date: **07-02-2026**

This is to certify that the Software Engineering Work entitled **"AUCTION MANAGEMENT SYSTEM"**, carried out by the group of students mentioned below under my guidance is approved for the Degree of Bachelor of Engineering (Semester-I) of L. J. Institute of Engineering and Technology (LJU) during the academic year 2025-26.

1. **ATIF (25002170210058)**
2. **KRISH (25002170210075)**
3. **RUTVI (25002170210026)**

Guide

Ms.Nidhi Mehta
FY2- Department, L.J.I.E.T.

Table of Contents

Sr. No.		Particulars	Page No.
1		INTRODUCTION	1
	1.1	Purpose	1
	1.2	Intended Audience	1
	1.3	Project Scope	1
	1.4	Glossary	2
	1.5	Overview	3
	1.6	Document Convection	3
2		OVERALL DESCRIPTION	4
	2.1	Product Perspective	4
	2.2	Product Features	5
	2.3	User Classes and Characteristics	6
	2.4	Operating Environment	7
	2.5	User Documentation	7
	2.6	Assumption and Dependencies	8
	2.7	Constraints	8
3		REQUIREMENT SPECIFICATION	9
	3.1	Functional Requirements	9
		3.1.1 Performance Requirements	9
		3.1.2 Design constraints	10
		3.1.3 Hardware Requirements	10
		3.1.4 Software Requirements	11
		3.1.5 Other Requirements	11
	3.2	Non-Functional Requirements	12
		3.2.1 Usability Requirements	12
		3.2.2 Security Requirements	12
		3.2.3 Reliability Requirements	13
		3.2.4 Maintainability Requirements	13
		3.2.5 Portability Requirements	13
		3.2.6 Scalability Requirements	13
		3.2.7 Availability Requirements	14

		3.2.8 Error Handling Requirements	**14**
		3.2.9 Data Integrity Requirements	**14**
		3.2.10 Legal and Ethical Requirements	**14**
4		**DIAGRAMS**	**15**
	4.1	**Use-Case Diagram**	**15**
	4.2	**DFD (Data Flow Diagram)**	**16**
	4.3	**Activity Diagram**	**17**
	4.4	**State Diagram**	**18**
	4.5	**Sequence Diagram**	**19**
	4.6	**Class Diagram**	**20**
5		**CONCLUSION**	**21**
6		**SCOPE OF THE PROJECT**	**22**

1. INTRODUCTION

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) document is to clearly explain the requirements and working of the Auction Management System.

This document describes:

- What the system does
- Who will use the system
- How the system behaves
- What features are included in the project

The Auction Management System is a Java-based console application that allows users to participate in auctions. It helps in managing users, auction items, bidding, payment, and earnings calculation.

This SRS document acts as a guide for understanding, developing, testing, and maintaining the system.

1.2 Intended Audience

This document is written for the following people:

- Students who are developing or studying the project.
- Teachers / Project evaluators who will assess the project.
- Developers who want to understand or modify the system.
- Testers who want to verify the system's correctness.
- Future users or maintainers of the application.

The document is written in simple language so that even users with basic programming knowledge can easily understand it.

1.3 Project Scope

The Auction Management System is designed to manage auction activities in a simple and controlled manner.

The system allows:

- Owner registration and login.
- Appointment of a seller by the owner.
- Adding auction items with a base price.
- Buyers to register and log in.
- Buyers to view available items.
- Buyers to place bids on open items.
- Owner to close auctions.

- Automatic winner selection.
- Buyers to make payment after winning.
- Calculation of earnings for owner and seller.

The system does NOT support:

- Graphical user interface (GUI).
- Database storage.
- Multiple sellers.
- Real-time auction timers.
- Mobile or web-based access.

This project is developed mainly for academic learning and focuses on demonstrating Object-Oriented Programming concepts in Java.

1.4 Glossary

TERM	MEANING
Auction	A process where items are sold to the person who bids the highest price
Owner	The main administrator of the system
Seller	The person who sells items through auction
Buyer	The person who bids and buys items
Bid	The price offered by a buyer for an item
Base Price	The minimum starting price of an item
Highest Bid	The maximum bid placed on an item
Auction Status	Shows whether the auction is open or closed
Payment	The process of paying the winning bid amount
Earning	Money earned by owner and seller
SRS	Software Requirements Specification
OOP	Object Oriented Programming
JDK	Java Devlopment Kit

1.5 Overview

This SRS document is divided into different sections to explain the system clearly:

- **Section 1** explains the introduction, purpose, scope, and document details.
- **Section 2** explains the overall system description.
- **Section 3** describes requirement specification, functional requirements and non-functional requirements like performance and security.
- **Section 4** there are 6 UML Diagram.
- **Section 5** gives the conclusion of this SRS.
- **Section 6** describes the scope of the project.

1.6 Document Covection

This document follows standard SRS writing rules:

- All headings are numbered for easy reference.
- Simple and clear English is used.
- Technical terms are explained in the glossary.
- Important words are written clearly to avoid confusion.
- For main headings font size 16 is used.
- For partial headings font size 14 is used.
- For contents font size 12 is used.
- Everything is written in the Times New Roman font style.

Technical Details of the Project:

- Programming Language: Java
- Application Type: Console-based application
- User Interface: Command Line Interface (CLI)
- Programming Style: Object-Oriented Programming (OOP)
- Data Storage: Arrays (In-memory storage)
- Platform Required: Any system with Java JDK 8 or above

2. OVERALL DESCRIPTION

2.1 Product Perspective

The Auction Management System is a standalone, menu-driven Java application developed using core Java concepts.
The system runs completely in the command line environment and does not require any external software, database, or internet connection.

The application follows an Object-Oriented Programming (OOP) structure.
Each real-world entity such as Owner, Seller, Buyer, and Item is represented using a separate Java class.

The main control of the system is handled by the Auction-system class, which manages:

- User interaction
- Menu display
- Input handling
- System flow control

All data is stored in fixed-size arrays, such as:

- Owner array (size 1)
- Seller array (size 1)
- Buyer array (size 20)
- Item array (size 20)

This design keeps the system simple and suitable for academic demonstration.The system uses Object-Oriented Programming (OOP) concepts like:

- Inheritance (User → Owner, Seller, Buyer)
- Encapsulation (data and methods inside classes)
- Abstraction (role-based access)
- Polymorphism (Method Overriding)

This system replaces the manual auction process by providing a structured way to:

- Register users
- Add auction items
- Place bids
- Declare winners
- Complete payments

The application works as a standalone system and does not require:

- Internet connection
- External database
- Third-party payment gateway

2.2 Product Features

User Management Features

- Strong password validation for all users.
- Secure login system for Owner, Seller, and Buyer.
- Unique role-based menus.
- Prevention of duplicate buyer registration.

Owner Features

- Owner registration and authentication.
- Appointment of a single seller.
- Add auction items with base price.
- Prevent duplicate item entries.
- View complete item list with status.
- Close auction manually.
- Declare auction winners.
- View list of registered buyers and total number of buyers registered.
- View total owner earnings.
- Automatic revenue calculation.

Seller Features

- Seller login and authentication.
- View all auction items.
- View auction status (open/closed).
- View seller earnings.
- View buyer details.
- Earn commission after successful payment.

Buyer Features

- Buyer registration with validation.
- Buyer login.
- View available items.
- Place bids on open auctions only and only when the seller is appointed.
- Validation of bid amount that placed bid should be greater than base price or current bid..
- Automatic update of highest bid.
- Make payment only after winning auction.

Auction Management Features

- Maintain auction status (Open / Closed).
- Allow bidding only when auction is open.
- Identify highest bidder.
- Declare winner after auction closure.
- Prevent multiple payments for same item.

Payment Features

- Display winning amount before payment.
- Multiple payment options:

    1. Cash
    2. Card
    3. Online (UPI)

- Validation of payment amount.
- Simulated payment process.
- Payment confirmation messages.
- Automatic earnings distribution.

2.3 User Classes and Characteristics

Owner

- Highest authority in the system.
- Controls auction operations.
- Manages sellers and items.
- Has administrative privileges.
- Expected to have basic computer skills.
- Responsible for closing auctions.

Seller

- Appointed by the owner.
- Cannot add items independently.
- Can monitor auction items.
- Earns commission after item sale.
- Limited system access.
- Uses system mainly for tracking earnings.

Buyer

- Main participant in auction process.
- Can register and log in independently.
- Can bid on auction items.
- Can make payments.
- Can view item details.
- Uses simple text menus.
- May have minimal technical knowledge.

2.4 Operating Environment

- Programming Language: Java.
- Execution Mode: Console-based.
- Supported Operating Systems:
  1. Windows
  2. Linux
  3. macOS
- Development Tool:

  IntelliJ IDEA
- Java Version Required:

  JDK 8 or above
- Hardware Requirements:
  1. Minimum 4 GB RAM
  2. Keyboard and Monitor
  3. Basic CPU

2.5 User Documentation

User documentation is provided in the form of:

- Console-based menus.
- Step-by-step instructions.
- Clear prompts for input.
- Error messages for invalid actions.
- Informative messages after every operation.

The system does not require separate manuals because:

- Navigation is menu-driven.
- Instructions are self-explanatory.
- Each operation is guided by printed messages.

2.6 Assumptions and Dependencies

Assumptions

- Only one owner and one seller exist.
- Buyers enter correct data.
- Users follow menu instructions.
- Payments are simulated.
- Auction items are limited.
- Internet connection is not required.
- System is used in controlled environment.

Dependencies

- Java Runtime Environment (JRE).
- Java Scanner class for input handling.
- Proper functioning of console I/O.
- Correct sequence of user actions.
- System depends on manual auction closure.

2.7 Constraints

- No real payment processing.
- No database integration.
- Data is temporary and non-persistent.
- Maximum number of buyers and items is fixed.
- Only one seller allowed.
- Also only one buyer.
- Console-based interface only.
- No graphical user interface.
- Password rules are predefined.
- System is not scalable for large users.

3. REQUIREMENT SPECIFICATION

The functional requirements describe the exact behavior and functionality of the Auction Management System. They explain what the system is expected to do, how users interact with it, and how the system responds to user actions.

These requirements are strictly derived from the implemented Java code and reflect the actual working of the system. The Auction Management System supports multiple user roles such as Owner, Seller, and Buyer. Each role is provided with a separate menu and restricted access, ensuring controlled and secure usage of the system.

3.1 Functional Requirements

This section describes the supporting functional requirements that help the system work properly. These requirements define the constraints, hardware, software, and other necessary conditions needed for the Auction Management System to function smoothly.

The system is designed as a console-based Java application that follows object-oriented programming principles. All requirements mentioned below are derived from the structure, logic, and behavior of the implemented code.

3.1.1 Performance Requirements

The performance requirements specify how efficiently the system should operate during normal usage. Since the system is console-based and runs locally, it is expected to provide quick responses to all user actions.

The system should respond immediately when a user selects any menu option.

Operations such as login, viewing items, placing bids, closing auctions, and making payments should execute without noticeable delay. Because all data is stored in memory using arrays, data retrieval and updates should be fast and reliable.

The auction process should work smoothly.

When a buyer places a bid, the system must instantly check whether the auction is open,if the seller is appointed or not and whether the bid amount is valid. Once the owner closes the auction, the system should immediately stop accepting bids and display the winning buyer details.

The payment process should also perform efficiently.

After selecting the payment option, the system should immediately validate the entered amount and confirm the payment. Earnings for the owner and seller should be calculated automatically and displayed without delay.

Overall, the system should ensure smooth execution, minimal memory usage, and stable performance throughout its operation.

3.1.2 Design Constraints

Design constraints are the limitations under which the system is developed. The Auction Management System is designed as a console-based application using Java. It does not include a graphical user interface and does not support real-time online bidding. All data is stored temporarily in memory using arrays, and the system operates in a single-user environment at a time.

The design also restricts the number of users and auction items due to the use of fixed-size arrays. Payment functionality is simulated and does not involve real money transactions. These constraints ensure that the system remains lightweight, easy to test, and suitable for academic demonstrations.

- System is console-based, not GUI-based
- Developed using Java programming language
- Uses text-based menus and keyboard input
- Data is stored temporarily using arrays
- No external database is used
- Limited number of users and items
- Single-user execution at a time
- No internet connection required
- No real-time bidding support
- Payment process is simulated

3.1.3 Hardware Requirements

The Auction Management System does not require high-end hardware because it is a lightweight, text-based application. It can run efficiently on basic computers or laptops. Since all processing is done locally and no graphics or heavy computations are involved, minimal system resources are sufficient for smooth operation.

The hardware requirements are kept low to ensure that the system can be easily executed on most student computers and lab machines without any performance issues.

- Computer or laptop required
- Minimum 2 GB RAM
- At least 200 MB free disk space
- Standard keyboard for input
- Display screen for output
- Basic processor (Intel i3 or equivalent recommended)
- Mouse is optional

3.1.4 Software Requirements

The system requires a Java-supported software environment to execute. Since the application is written in Java, the Java Development Kit (JDK) and Java Runtime Environment (JRE) are mandatory. The system is platform-independent and can run on any operating system that supports Java.

Only standard Java libraries are used, which eliminates the need for third-party frameworks or external dependencies. This makes the system easy to install, execute, and maintain.

- Operating system:
  1. Windows
  2. Linux
  3. macOS
- Java Development Kit (JDK) version 8 or higher
- Java Runtime Environment (JRE)
- Any Java IDE (Eclipse, IntelliJ, NetBeans, VS Code)
- Command Prompt or Terminal

3.1.5 Other Requirements

Other requirements include usability, data handling, operational behavior, documentation, and ethical considerations. The system should be easy to use, even for beginners. Menus and instructions should be clear, and error messages should help users correct mistakes.

The system must handle data securely during execution and prevent invalid actions such as incorrect bidding or duplicate payments. Proper documentation and clean code structure are required for maintainability. Since the project is academic, all transactions are simulated and no personal data is stored permanently.

User Interface Requirements

- Clear menu-driven interface
- Numbered options for easy selection
- Step-by-step instructions
- Simple and meaningful error messages

Data Handling Requirements

- Temporary storage of user data
- Proper validation of bid amounts
- Correct tracking of highest bids
- Payment allowed only for winning buyer

Operational Requirements

- User must log in before accessing features
- Auction must be opened before bidding
- Auction must be closed to declare winner
- Payment allowed only after auction completion

Documentation Requirements

- Well-commented code
- Meaningful class and method names
- Proper project documentation

Legal and Ethical Requirements

- No real money transactions
- System used only for educational purposes
- User data not stored permanently

3.2 Non-Functional Requirements

Non-functional requirements define the quality characteristics of the system.

They describe how well the system performs its functions, rather than what functions it performs.

These requirements focus on usability, security, reliability, maintainability, and other important system qualities.

3.2.1 Usability Requirements

The Auction Management System should be simple and easy to use for all users. The interface is menu-driven, allowing users to navigate through numbered options easily.

Each menu displays clear instructions, helping users understand what action to perform next.Error messages are simple and informative, guiding users whenever an invalid input is entered.

The system does not require prior technical knowledge.Even first-time users can operate the system by following the on-screen instructions. Consistency in menu design ensures that users feel comfortable while navigating different sections.

3.2.2 Security Requirements

Security is an important requirement of the system. All users must create strong passwords that include letters, numbers, and special characters.

Login authentication ensures that only authorized users can access the system.Each user role has restricted access, preventing unauthorized actions. Buyers cannot access seller or owner functions, and only the owner can appoint sellers.

The system also ensures payment security by allowing only the winning buyer to make a payment.Payment amounts are validated to prevent incorrect or unauthorized transactions.

3.2.3 Reliability Requirements

The system should operate correctly under normal conditions. It must handle incorrect inputs gracefully without crashing.

The system should prevent duplicate entries such as repeated buyers or duplicate items. Invalid bids, incorrect payments, and unauthorized access attempts should be rejected properly.

Once a payment is completed, the system should not allow the same item to be paid for again. Auction data such as bids, payment status, and earnings should remain accurate throughout execution.

3.2.4 Maintainability Requirements

The Auction Management System is designed using Object-Oriented Programming principles, making it easy to maintain. Each class has a specific responsibility, reducing code complexity.

The modular structure allows developers to modify or extend features easily. Future improvements such as database integration or graphical interface can be added without rewriting the entire system.

Readable code structure and method separation make debugging and updates simple and efficient.

3.2.5 Portability Requirements

The system should be platform independent.Since it is developed using Java, it can run on any operating system that supports Java Virtual Machine (JVM).

No operating-system-specific libraries are used.The system requires only a Java SE environment, making it easy to deploy on different machines.

3.2.6 Scalability Requirements

The current system is designed for academic purposes and supports a limited number of users. Arrays are used for storing buyers and items, which limits scalability.

However, the system can be enhanced by replacing arrays with dynamic data structures or databases. Additional user roles and advanced auction features can be introduced in future versions.

3.2.7 Availability Requirements

The system should be available whenever it is executed. No internet connection is required, making it suitable for offline usage.

The system depends only on the Java runtime environment. As long as Java is installed, the system will function correctly.

3.2.8 Error Handling Requirements

The system should handle errors gracefully. Invalid menu selections should display appropriate error messages.

Wrong login credentials, invalid bids, incorrect payment amounts, and invalid item IDs should be detected and reported clearly. The system should continue running even after encountering errors, ensuring uninterrupted user interaction.

3.2.9 Data Integrity Requirements

The system should maintain accurate and consistent data. Bid values, payment status, auction status, and earnings must be updated correctly.

No conflicting or incorrect data should be stored during execution. Each operation should reflect correct system state at all times.

3.2.10 Legal and Ethical Requirements

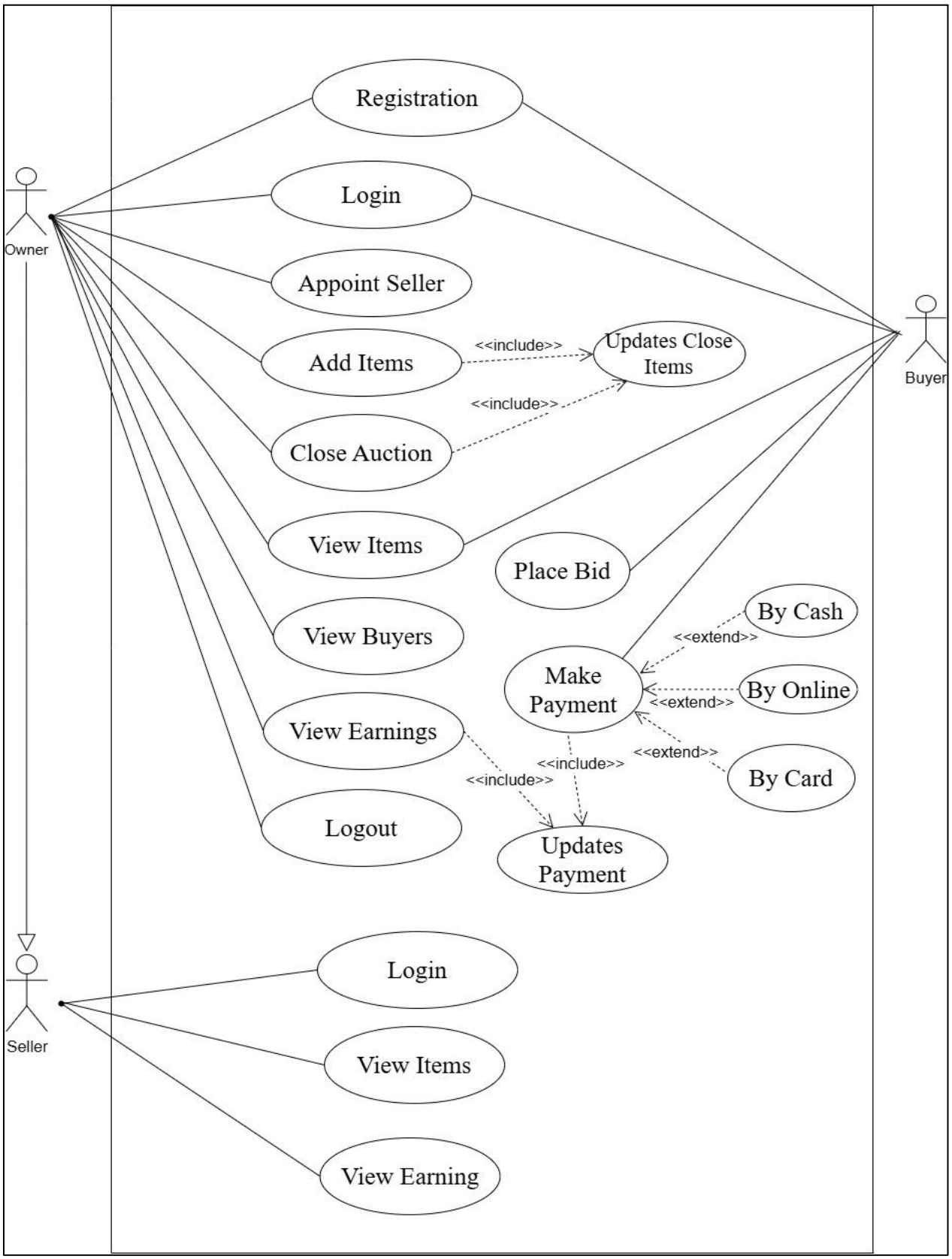
The Auction Management System is developed strictly for educational purposes. No real money transactions are involved.

All payments are simulated, and no sensitive user data is stored permanently. The system does not violate any legal or ethical standards.

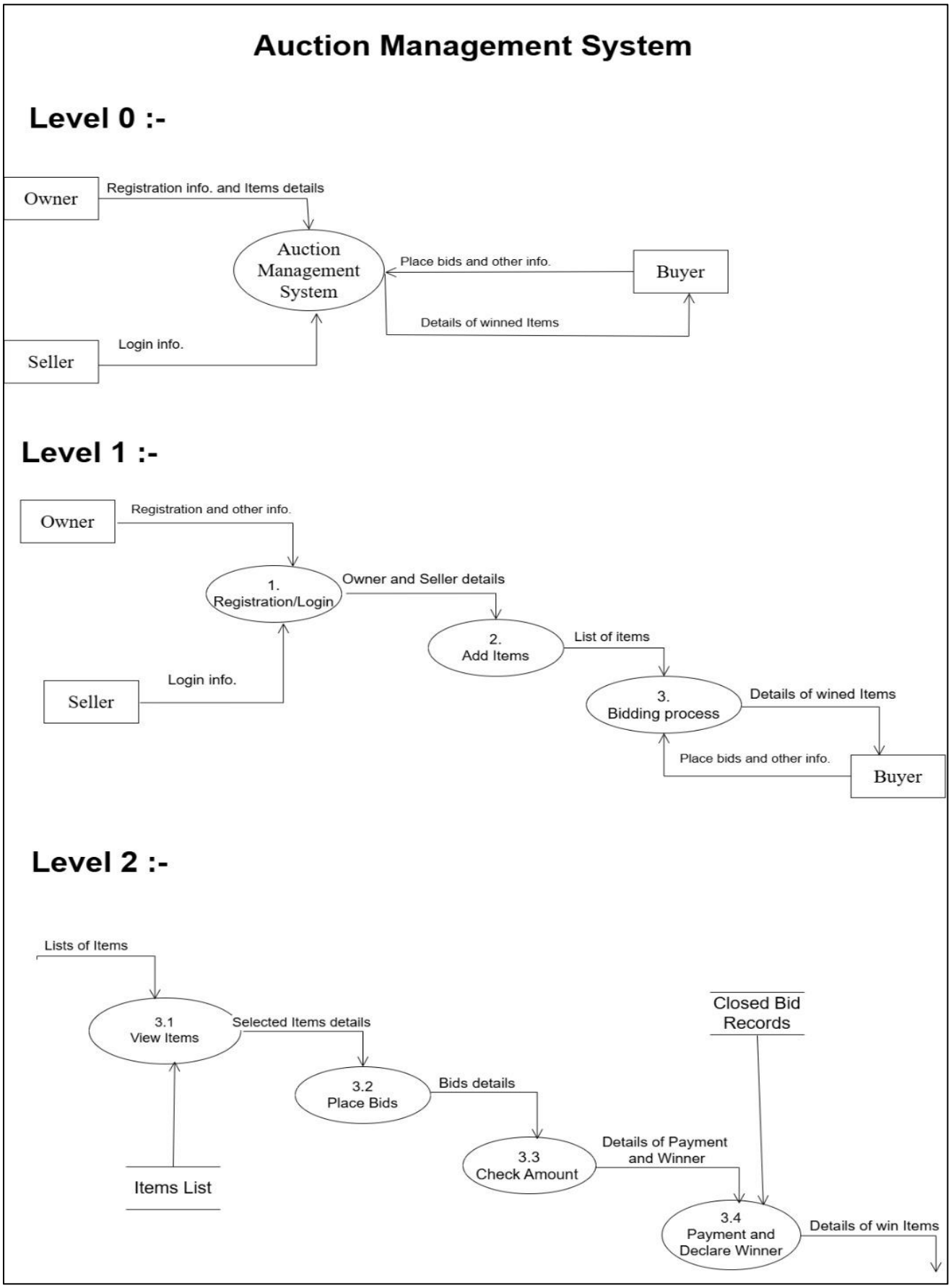
4. DIAGRAMS

In this section there are the 6 UML (Unified Modeling Language) to be shown:

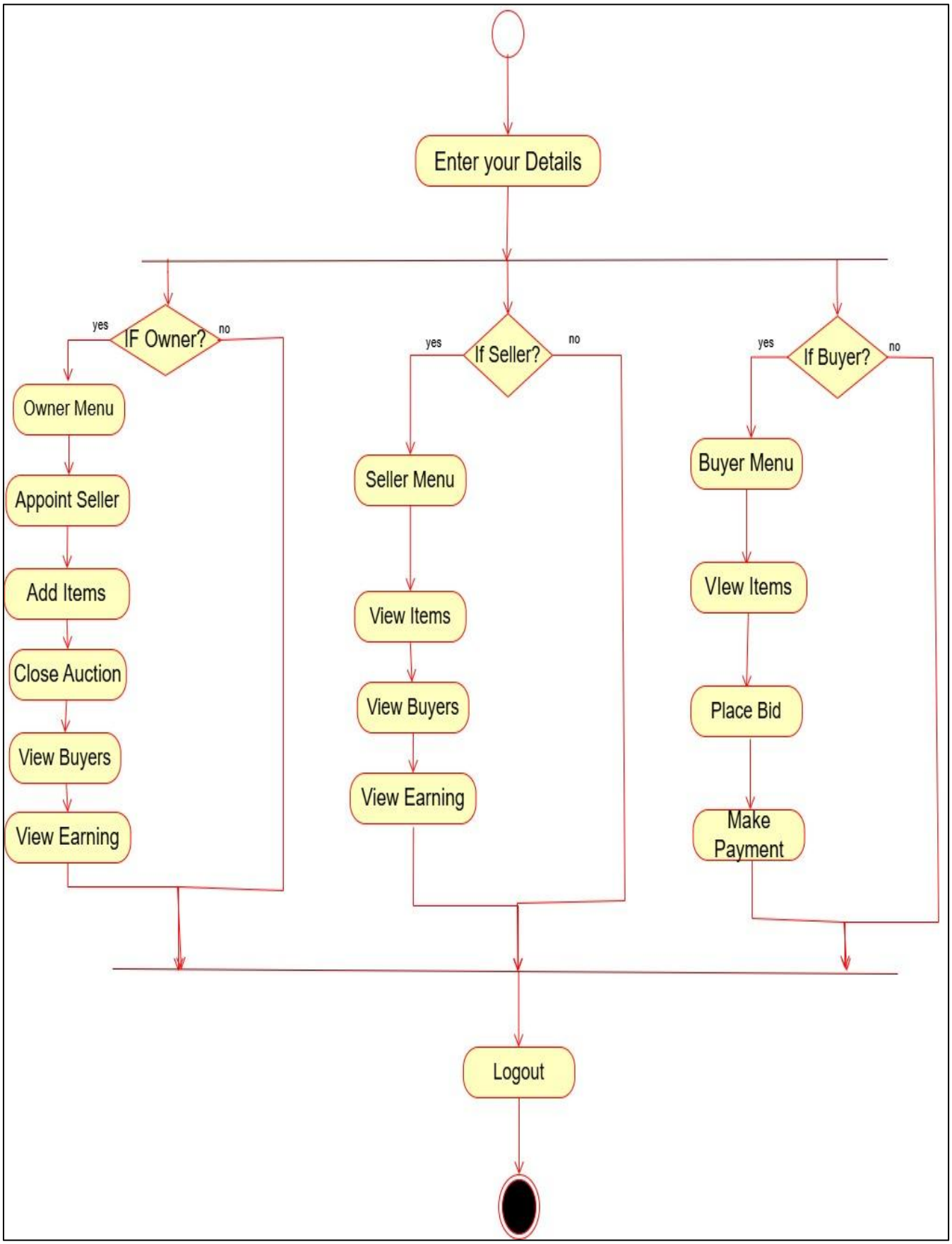
4.1 Use-Case Diagram



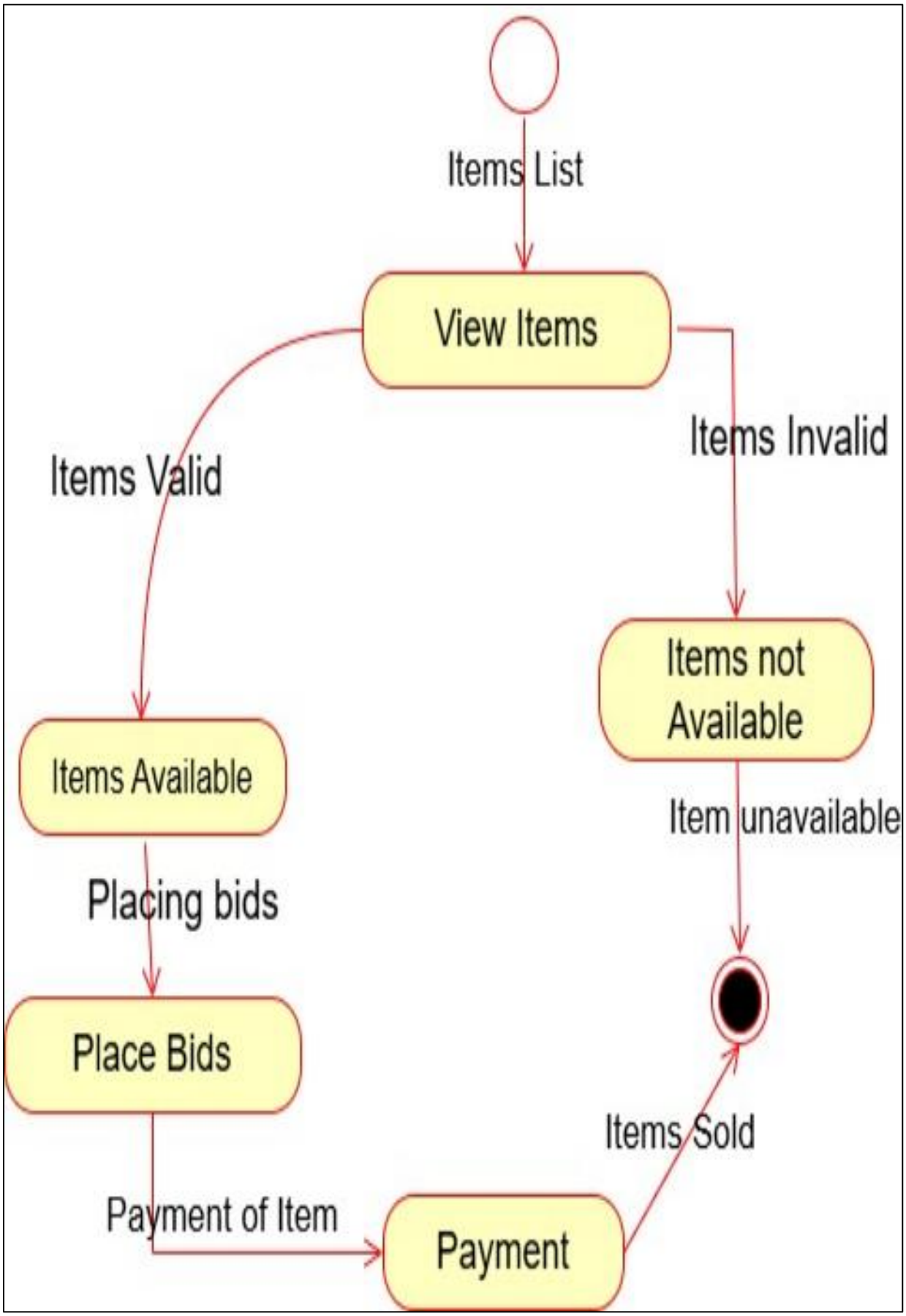
4.2 DFD (Data Flow Diagram)



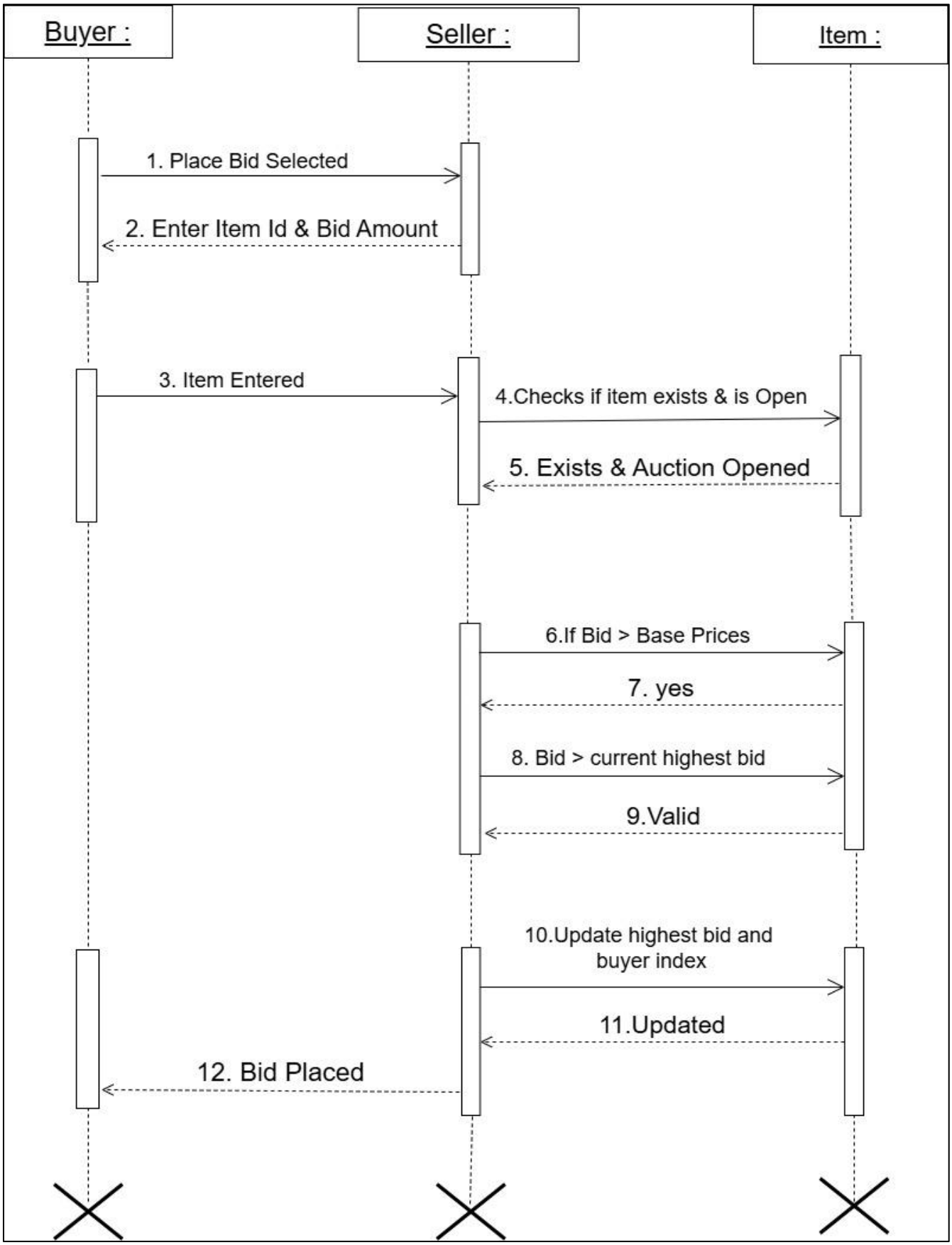
4.3 Activity Diagram



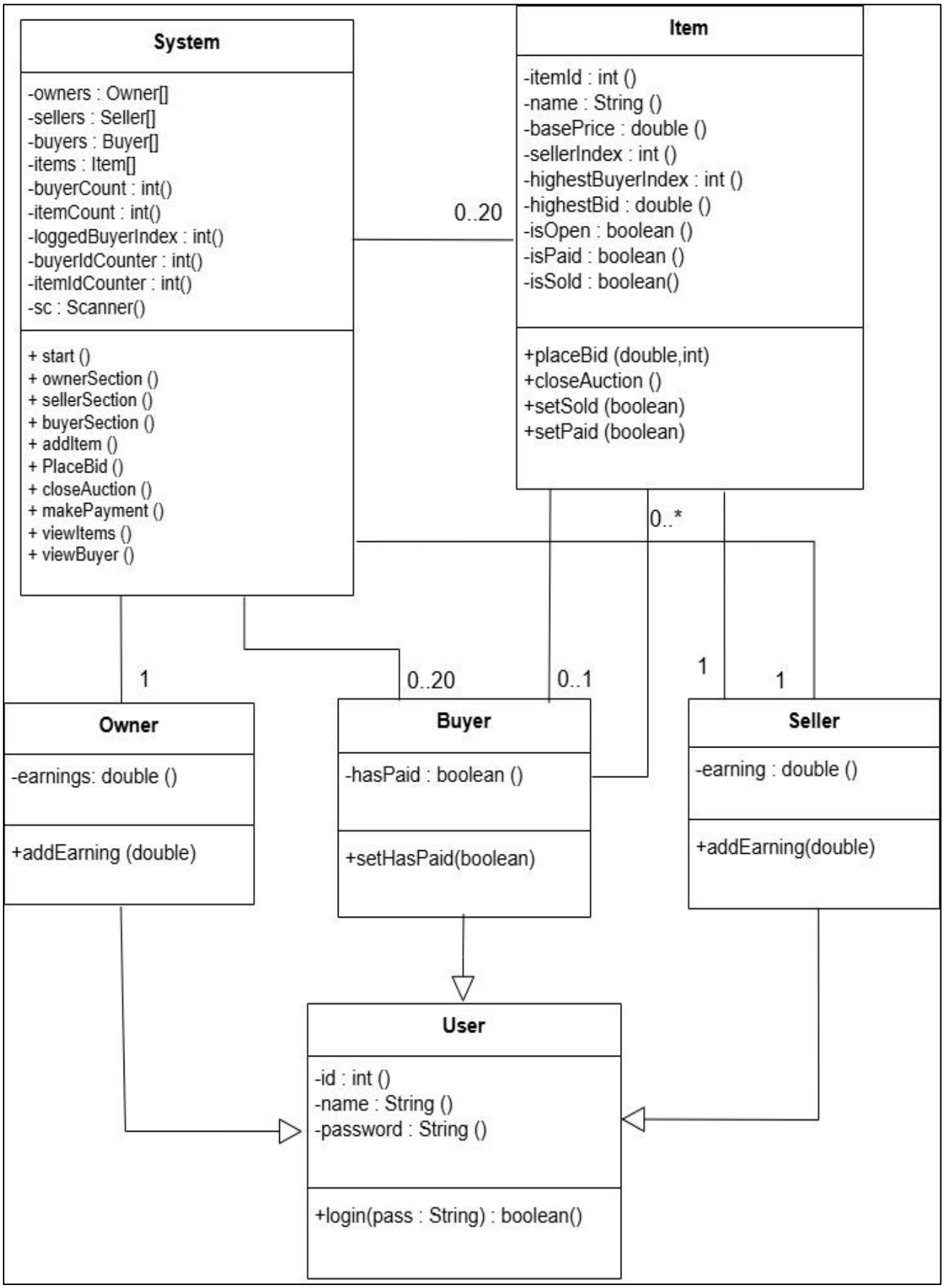
4.4 State Diagram



4.5 Sequence Diagram



4.6 Class Diagram



5. CONCLUSION

The Auction Management System developed using Java is a simple and effective console-based application that successfully demonstrates how an auction process works. The system clearly separates users into owner, seller, and buyer roles, allowing each user to perform specific actions such as adding items, placing bids, closing auctions, and making payments. It correctly handles bidding rules by allowing bids only when auctions are open and ensuring that each new bid is higher than the current bid or base price. The system automatically identifies the winning buyer after an auction is closed and supports different payment methods such as cash, card, and online payment in a simulated manner. By using object-oriented programming concepts like inheritance, encapsulation, and modular design, the code is well-structured and easy to understand. Although the system uses arrays instead of a database and works only in a single-user environment, it fulfills the project objectives and serves as a strong academic model for understanding Java programming, system design, and auction management logic.

6. SCOPE OF THE PROJECT

The scope of the Auction Management System is to provide a simple and organized way to manage auction activities using a Java-based console application. The project focuses on demonstrating how an auction process works, starting from user registration to bidding, auction closing, and payment handling. It is mainly designed for learning and academic purposes, where the goal is to understand object-oriented programming concepts and basic system design rather than building a commercial application.

The system covers multiple user roles such as Owner, Seller, and Buyer, each having clearly defined responsibilities. The owner can appoint a seller, add auction items, close auctions, and view total earnings. The seller can view items, monitor buyers, and check earnings generated through successful auctions. Buyers can register, log in, view available items, place bids, and make payments after winning an auction. This clearly shows how role-based access works within a software system.

The project scope also includes bid management, where buyers can place bids only if the auction is open and the bid amount is higher than the base price or current highest bid. This ensures fair bidding and prevents invalid inputs. Once the auction is closed, the system identifies the highest bidder automatically, reducing manual calculation and errors.

Payment processing is included in a simulated form. The system allows the winning buyer to make payment using different payment modes such as cash, card, or online payment. Although real payment gateways are not used, the logic demonstrates how payment validation and confirmation can be implemented in software systems.

The data used in the system is stored temporarily using arrays, which means all data is lost once the program stops. This keeps the system simple and easy to understand, making it suitable for beginners. The project does not include advanced features such as database storage, graphical user interface, or real-time multiple user access, which clearly defines its current scope and limitations.

Overall, the scope of this project is limited to a single-user, console-based auction management system that helps students understand Java programming, object-oriented concepts, and basic system workflows. The project can be further expanded in the future by adding databases, user authentication improvements, graphical interfaces, and online payment integration, but the current scope is well-suited for educational and learning purposes.